

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

Navya Pandey (1BF24CS196)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum) Department
of Computer Science and Engineering**



This is to certify that the Lab work entitled **“OPERATING SYSTEMS – 23CS4PCOPS”** carried out by **Navya Pandey (1BF24CS196)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS- (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni

Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda

Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-fit b) Best-fit c) First-fit	
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	

Course outcomes:

CO1	Apply the different concepts and functionalities of Operating System.
CO2	Analyse various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating System.
CO4	Conduct practical experiments to implement the functionalities of Operating system.

Lab program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

- a) FCFS
- b) SJF
- c) Priority
- d) Round Robin (Experiment with different quantum sizes for RR algorithm)

Code in C FCFS

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    int at[20], bt[20], ct[20], wt[20], tat[20], rt[20];
```

```
    float avg_wt=0, avg_tat=0, avg_rt=0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i=0;i<n;i++) {
```

```
        printf("Enter Arrival Time and Burst Time for Process %d: ", i+1);
```

```
        scanf("%d %d",&at[i],&bt[i]);
```

```
    }
```

```
    ct[0] = at[0] + bt[0];
```

```
    for(i=1;i<n;i++) {
```

```
        if(ct[i-1] < at[i])
```

```
            ct[i] = at[i] + bt[i];
```

```
        else
```

```
            ct[i] = ct[i-1] + bt[i];
```

```
    }
```

```

for(i=0;i<n;i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = wt[i]; // Response Time

    avg_wt += wt[i];
    avg_tat += tat[i];
    avg_rt += rt[i];
}

printf("\nP\tAT\tBT\tCT\tWT\tTAT\tRT\n");

for(i=0;i<n;i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], wt[i], tat[i], rt[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
printf("\nAverage Response Time = %.2f\n", avg_rt/n);

return 0;
}

```

Output

```
Enter number of processes: 3
Enter Arrival Time and Burst Time for Process 1: 0
7
Enter Arrival Time and Burst Time for Process 2: 2
4
Enter Arrival Time and Burst Time for Process 3: 4
1

P      AT      BT      CT      WT      TAT      RT
P1      0       7       7       0       7       0
P2      2       4      11       5       9       5
P3      4       1      12       7       8       7

Average Waiting Time = 4.00
Average Turnaround Time = 8.00
Average Response Time = 4.00

Process returned 0 (0x0)   execution time : 18.008 s
Press any key to continue.
|
```

Code in C SJF(non preemptive)

```
#include <stdio.h>

int main() {
    int n, i, j, time = 0, min, pos;
    int at[20], bt[20], ct[20], wt[20], tat[20], rt[20], done[20] = {0};
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for Process %d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    int completed = 0;

    while(completed < n) {
        min = 9999;
        pos = -1;

        for(i = 0; i < n; i++) {
            if(at[i] <= time && done[i] == 0) {
                if(bt[i] < min) {
                    min = bt[i];
                    pos = i;
                }
            }
        }

        if(pos != -1) {
            time += bt[pos];
            done[pos] = 1;
            completed++;
            wt[pos] = time - at[pos];
            tat[pos] = time;
            rt[pos] = time - bt[pos];
        }
    }

    avg_wt = (float)wt[0] / n;
    avg_tat = (float)tat[0] / n;
    avg_rt = (float)rt[0] / n;

    printf("Average waiting time: %f\n", avg_wt);
    printf("Average turnaround time: %f\n", avg_tat);
    printf("Average response time: %f\n", avg_rt);
}
```

```
    if(pos == -1) {
        time++;
    } else {
        ct[pos] = time + bt[pos];
        time = ct[pos];

        tat[pos] = ct[pos] - at[pos];
        wt[pos] = tat[pos] - bt[pos];
        rt[pos] = wt[pos];

        avg_wt += wt[pos];
        avg_tat += tat[pos];
        avg_rt += rt[pos];

        done[pos] = 1;
        completed++;
    }
}

printf("\nP\tAT\tBT\tCT\tWT\tTAT\tRT\n");

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], wt[i], tat[i], rt[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
```

```
printf("\nAverage Response Time = %.2f\n", avg_rt/n);
```

```
return 0;
```

```
}
```

OUTPUT

```
C:\Users\BMSCECSE\Desktop\ X + v
Enter number of processes: 3
Enter Arrival Time and Burst Time for Process 1: 0
7
Enter Arrival Time and Burst Time for Process 2: 2
4
Enter Arrival Time and Burst Time for Process 3: 4
1
P      AT      BT      CT      WT      TAT      RT
P1      0       7       7       0       7       0
P2      2       4      12       6      10       6
P3      4       1       8       3       4       3

Average Waiting Time = 3.00
Average Turnaround Time = 7.00
Average Response Time = 3.00

Process returned 0 (0x0)   execution time : 12.144 s
Press any key to continue.
|
```

Code in C SJF(preemptive)

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n, i, time = 0, completed = 0, min_bt, pos;
    int at[20], bt[20], rt[20], ct[20], wt[20], tat[20], start[20];
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for Process %d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];    // remaining time
        start[i] = -1;    // to track first start
    }

    while(completed < n) {
        min_bt = INT_MAX;
        pos = -1;

        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && rt[i] < min_bt) {
                min_bt = rt[i];
                pos = i;
            }
        }

        if(pos == -1) {
            time++;
            continue;
        }

        if(start[pos] == -1)
            start[pos] = time;    // first response time

        rt[pos]--;
        time++;

        if(rt[pos] == 0) {
            completed++;
            ct[pos] = time;
            tat[pos] = ct[pos] - at[pos];
            wt[pos] = tat[pos] - bt[pos];
        }
    }

    printf("\nP\tAT\tBT\tCT\tWT\tTAT\tRT\n");
```

```

for(i = 0; i < n; i++) {
    int response = start[i] - at[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
    avg_rt += response;

    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], wt[i], tat[i], response);
}

printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
printf("\nAverage Response Time = %.2f\n", avg_rt/n);

return 0;
}

```

OUTPUT

```

Enter number of processes: 3
Enter Arrival Time and Burst Time for Process 1: 0
7
Enter Arrival Time and Burst Time for Process 2: 2
4
Enter Arrival Time and Burst Time for Process 3: 4
1

P      AT      BT      CT      WT      TAT      RT
P1      0       7      12       5      12       0
P2      2       4       7       1       5       0
P3      4       1       5       0       1       0

Average Waiting Time = 2.00
Average Turnaround Time = 6.00
Average Response Time = 0.00

Process returned 0 (0x0)   execution time : 14.011 s
Press any key to continue.

```

Code in C Priority

```
#include <stdio.h>

struct Process {
    int at, bt, pr, rt, ct, tat, wt, done;
} p[20];

int main() {
    int n, i, time = 0, completed = 0, choice;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input processes clearly
    for (i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Priority (smaller number = higher priority): ");
        scanf("%d", &p[i].pr);

        p[i].rt = p[i].bt;
        p[i].done = 0;
    }

    printf("\nChoose Scheduling Type:\n");
    printf("1. Preemptive Priority\n");
    printf("2. Non-Preemptive Priority\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    while (completed < n) {
        int idx = -1, hp = 999;

        // Find the highest priority process that has arrived
        for (i = 0; i < n; i++) {
            if (p[i].at <= time && !p[i].done) {
                if (p[i].pr < hp) {
                    hp = p[i].pr;
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            time++;
            continue;
        }
    }
}
```

```

if (choice == 1) { // Preemptive
    p[idx].rt--;
    time++;

    if (p[idx].rt == 0) {
        p[idx].done = 1;
        completed++;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
    }
} else { // Non-Preemptive
    time += p[idx].bt;
    p[idx].done = 1;
    completed++;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
}
}

float awt = 0, atat = 0;

// Print table header
printf("\n%-5s %-5s %-5s %-8s %-5s %-5s %-5s\n",
        "PID", "AT", "BT", "PR", "CT", "TAT", "WT");

for (i = 0; i < n; i++) {
    printf("%-5d %-5d %-5d %-8d %-5d %-5d %-5d\n",
            i + 1, p[i].at, p[i].bt, p[i].pr,
            p[i].ct, p[i].tat, p[i].wt);

    awt += p[i].wt;
    atat += p[i].tat;
}

printf("\nAverage Waiting Time = %.2f", awt / n);
printf("\nAverage Turnaround Time = %.2f\n", atat / n);

return 0;

```

OUTPUT

Enter number of processes: 3

Process 1

Arrival Time: 4

Burst Time:

6

Priority (smaller number = higher priority): 5

Process 2

Arrival Time: 3

Burst Time: 8

Priority (smaller number = higher priority): 4

Process 3

Arrival Time: 2

Burst Time: 7

Priority (smaller number = higher priority): 4

Choose Scheduling Type:

1. Preemptive Priority

2. Non-Preemptive Priority

Enter choice: 2

PID	AT	BT	PR	CT	TAT	WT
1	4	6	5	23	19	13
2	3	8	4	17	14	6
3	2	7	4	9	7	0

Average Waiting Time = 6.33

Average Turnaround Time = 13.33

Process returned 0 (0x0) execution time : 11.914 s

Press any key to continue.

Enter number of processes: 3

Process 1

Arrival Time: 5

Burst Time: 43

Priority (smaller number = higher priority): 2

Process 2

Arrival Time: 7

Burst Time: 4

Priority (smaller number = higher priority): 9

Process 3

Arrival Time: 3

Burst Time: 65

Priority (smaller number = higher priority): 4

Choose Scheduling Type:

1. Preemptive Priority

2. Non-Preemptive Priority

Enter choice: 1

PID	AT	BT	PR	CT	TAT	WT
1	5	43	2	48	43	0
2	7	4	9	115	108	104
3	3	65	4	111	108	43

Average Waiting Time = 49.00

Average Turnaround Time = 86.33

Process returned 0 (0x0) execution time : 10.041 s

Press any key to continue.

Code in C Round Robin

```
#include <stdio.h>

struct process{
    int pid;
    int at, bt;
    int ct, tat, wt, rt;
    int remaining_bt;
    int first_exec;
};

int main(){

    int n, tq, time=0, completed=0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct process p[n];

    for(int i=0; i<n; i++){
        p[i].pid=i+1;

        printf("Enter AT and BT for P%d: ", p[i].pid);
        scanf("%d %d", &p[i].at, &p[i].bt);

        p[i].remaining_bt=p[i].bt;
        p[i].first_exec=-1;
    }
```

```

printf("Enter Time Quantum: ");
scanf("%d",&tq);

while(completed<n){

    int executed=0;

    for(int i=0;i<n;i++){

        if(p[i].at<=time && p[i].remaining_bt>0){

            if(p[i].first_exec==-1){
                p[i].rt=time-p[i].at;
                p[i].first_exec=1;
            }

            if(p[i].remaining_bt>tq){
                time+=tq;
                p[i].remaining_bt-=tq;
            }
            else{
                time+=p[i].remaining_bt;
                p[i].remaining_bt=0;

                p[i].ct=time;
                p[i].tat=p[i].ct-p[i].at;
                p[i].wt=p[i].tat-p[i].bt;

                completed++;
            }
        }
    }
}

```

```

        executed=1;
    }
}

if(!executed){
    time++;
}
}

float avg_tat=0,avg_wt=0,avg_rt=0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i=0;i<n;i++){

    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,p[i].at,p[i].bt,
        p[i].ct,p[i].tat,p[i].wt,p[i].rt);

    avg_tat+=p[i].tat;
    avg_wt+=p[i].wt;
    avg_rt+=p[i].rt;
}

printf("\nAverage TAT = %.2f",avg_tat/n);
printf("\nAverage WT = %.2f",avg_wt/n);
printf("\nAverage RT = %.2f\n",avg_rt/n);
return 0;
}

```

OUTPUT

```
Enter number of processes: 3
Enter AT and BT for P1: 2
4
Enter AT and BT for P2: 6
9
Enter AT and BT for P3: 2
8
Enter Time Quantum: 2

PID      AT      BT      CT      TAT      WT      RT
P1        2        4        8        6        2        0
P2        6        9       23       17        8        2
P3        2        8       20       18       10        2

Average TAT = 13.67
Average WT  = 6.67
Average RT  = 1.33

Enter number of processes: 3
Enter AT and BT for P1: 2
4
Enter AT and BT for P2: 6
9
Enter AT and BT for P3: 2
8
Enter Time Quantum: 3

PID      AT      BT      CT      TAT      WT      RT
P1        2        4        9        7        3        0
P2        6        9       23       17        8        3
P3        2        8       20       18       10        3

Average TAT = 14.00
Average WT  = 7.00
Average RT  = 2.00

Process returned 0 (0x0)  execution time : 8.755 s
Press any key to continue.
```

Lab program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code in C

```
#include <stdio.h>

struct process{
    int id;
    int at, bt;
    int ct, wt, tat;
    int type;
    int rt;
};

int main(){

    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct process p[n];

    for(int i=0; i<n; i++){

        printf("\nProcess %d\n", i);

        p[i].id=i;

        printf("Enter arrival time: ");
        scanf("%d", &p[i].at);

        printf("Enter burst time: ");
        scanf("%d", &p[i].bt);

        printf("Enter type (0 = System, 1 = User): ");
        scanf("%d", &p[i].type);

        p[i].rt=p[i].bt;
    }

    int completed=0, time=0;

    while(completed<n){

        int idx=-1;

        for(int i=0; i<n; i++){
            if(p[i].at<=time && p[i].rt>0 && p[i].type==0){
                if(idx==-1 || p[i].at<p[idx].at)
```

```

        idx=i;
    }
}

if(idx== -1){
    for(int i=0;i<n;i++){
        if(p[i].at<=time && p[i].rt>0 && p[i].type==1){
            if(idx== -1 || p[i].at<p[idx].at)
                idx=i;
        }
    }
}

if(idx== -1){
    time++;
    continue;
}

p[idx].rt--;
time++;

if(p[idx].rt==0){

    p[idx].ct=time;
    p[idx].tat=p[idx].ct-p[idx].at;
    p[idx].wt=p[idx].tat-p[idx].bt;

    completed++;
}
}

float avgwt=0,avgtat=0;

printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");

for(int i=0;i<n;i++){

    if(p[i].type==0)
        printf("%d\tSystem\t%d\t%d\t%d\t%d\t%d\n",p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].wt,p[i].tat);
    else
        printf("%d\tUser\t%d\t%d\t%d\t%d\t%d\n",p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].wt,p[i].tat);

    avgwt+=p[i].wt;
    avgtat+=p[i].tat;
}

printf("\nAverage Waiting Time: %.2f",avgwt/n);
printf("\nAverage Turnaround Time: %.2f\n",avgtat/n);

return 0;
}

```

OUTPUT

Enter number of processes: 3

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = System, 1 = User): 0

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
1	User	2	3	12	7	10
2	System	1	4	9	4	8

Average Waiting Time: 3.67

Average Turnaround Time: 7.67

Process returned 0 (0x0) execution time : 22.999 s

Press any key to continue.

Lab program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

Code in C

```
#include <stdio.h>
#include <math.h>

#define MAX 10

typedef struct {
    int id, bt, deadline, period;
    int ct, wt, tat;
} Process;

void sortEDF(Process p[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=0;j<n-i-1;j++) {
            if(p[j].deadline > p[j+1].deadline) {
                Process temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
}

void sortRMS(Process p[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=0;j<n-i-1;j++) {
            if(p[j].period > p[j+1].period) {
                Process temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
}
```

```

float calculateUtilization(Process p[], int n) {
    float u = 0;
    for(int i=0;i<n;i++) {
        u += (float)p[i].bt / p[i].period;
    }
    return u;
}

void calculateTimes(Process p[], int n) {
    int time = 0;
    for(int i=0;i<n;i++) {
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct;
        p[i].wt = p[i].tat - p[i].bt;
    }
}

void printTable(Process p[], int n, int isEDF) {
    printf("ID\tBF\t");
    if(isEDF) printf("Deadline\t");
    else printf("Period\t");
    printf("CT\tWT\tTAT\n");

    for(int i=0;i<n;i++) {
        printf("%d\t%d\t", p[i].id, p[i].bt);
        if(isEDF) printf("%d\t", p[i].deadline);
        else printf("%d\t", p[i].period);
        printf("%d\t%d\t%d\n", p[i].ct, p[i].wt, p[i].tat);
    }
}

void proportionalScheduling(Process p[], int n) {
    printf("\n===== Proportional Scheduling =====\n");

    float total = 0;
    for(int i=0;i<n;i++) total += p[i].bt;

    printf("ID\tBurst\tShare\n");
    for(int i=0;i<n;i++) {
        float share = p[i].bt / total;
        printf("%d\t%d\t%.2f\n", p[i].id, p[i].bt, share);
    }
}

```

```

}

int main() {
    int n;
    Process p[MAX], temp[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i=0;i<n;i++) {
        p[i].id = i;
        printf("\nProcess %d:\n", i);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Deadline (EDF): ");
        scanf("%d", &p[i].deadline);
        printf("Period (RMS): ");
        scanf("%d", &p[i].period);
    }

    // Copy array for EDF
    for(int i=0;i<n;i++) temp[i] = p[i];

    printf("\n===== Earliest Deadline First (EDF) =====\n");
    sortEDF(temp, n);
    calculateTimes(temp, n);

    float u = calculateUtilization(temp, n);
    printf("CPU Utilization: %.2f\n", u);
    if(u <= 1) printf("Schedulable (Utilization <= 1)\n");

    printTable(temp, n, 1);

    // Copy again for RMS
    for(int i=0;i<n;i++) temp[i] = p[i];

    printf("\n===== Rate Monotonic Scheduling (RMS) =====\n");
    sortRMS(temp, n);
    calculateTimes(temp, n);

    float bound = n * (pow(2, (float)1/n) - 1);

    printf("CPU Utilization: %.2f\n", u);

```

```
printf("RM Bound: %.4f\n", bound);

if(u <= bound) printf("Schedulable (Utilization <= RM Bound)\n");

printTable(temp, n, 0);

// Proportional Scheduling
proportionalScheduling(p, n);

return 0;
}
```

OUTPUT

```
Enter number of users: 2

Enter user name: user1
Enter number of files for user1: 2
Enter file names:
a.txt
b.txt

User: user1
a.txt
b.txt

Enter user name: user2
Enter number of files for user2: 1
Enter file names:
x.txt

User: user2
x.txt
```

Lab program 4:

Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Code in C Producer Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;

void* producer(void* arg) {
    int item;
    for (int i = 0; i < 15; i++) {
        item = i;

        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 15; i++) {

        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
```

```

        sem_post(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}

```

OUTPUT

[PROBLEMS](#)
[OUTPUT](#)
[DEBUG CONSOLE](#)
[TERMINAL](#)
[PORTS](#)

```

Produced: 0 at buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Produced: 4 at buffer[4]
Consumed: 0 from buffer[0]
Consumed: 1 from buffer[1]
Consumed: 2 from buffer[2]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Consumed: 4 from buffer[4]
Produced: 7 at buffer[2]
Consumed: 5 from buffer[0]
Produced: 8 at buffer[3]
Consumed: 6 from buffer[1]

```

Code in C Dining Philosopher

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t forks[N];

void* philosopher(void* num) {
    int id = *(int*)num;

    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        if (id == N - 1) {
            // Prevent deadlock
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked right fork %d.\n", id, (id + 1) % N);

            sem_wait(&forks[id]);
            printf("Philosopher %d picked left fork %d.\n", id, id);
        } else {
            sem_wait(&forks[id]);
            printf("Philosopher %d picked left fork %d.\n", id, id);

            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked right fork %d.\n", id, (id + 1) % N);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(1);

        sem_post(&forks[id]);
        sem_post(&forks[(id + 1) % N]);

        printf("Philosopher %d put down forks %d and %d.\n",
            id, id, (id + 1) % N);
    }
}

int main() {
    pthread_t ph[N];
    int ids[N];
```

```
for (int i = 0; i < N; i++)
    sem_init(&forks[i], 0, 1);

for (int i = 0; i < N; i++) {
    ids[i] = i;
    pthread_create(&ph[i], NULL, philosopher, &ids[i]);
}

for (int i = 0; i < N; i++)
    pthread_join(ph[i], NULL);

return 0;
}
```

OUTPUT

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 3 is thinking.
Philosopher 0 picked left fork 0.
Philosopher 2 picked left fork 2.
Philosopher 2 picked right fork 3.
Philosopher 1 picked left fork 1.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked left fork 3.
Philosopher 3 picked right fork 4.
Philosopher 3 is eating.
Philosopher 1 picked right fork 2.
Philosopher 1 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 0 picked right fork 1.
```

Lab program 5:

Write a C program to simulate: (Any one)

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

Code in C Banker's algorithm

```
#include<stdio.h>
```

```
int main() {
    int n,m,i,j,k,c=0,f[10]={0},a[10][10],max[10][10],need[10][10],av[10],s[10];

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter number of resources: ");
    scanf("%d",&m);

    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&a[i][j]);

    printf("\nEnter Maximum Demand Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++) {
            scanf("%d",&max[i][j]);
            need[i][j]=max[i][j]-a[i][j];
        }

    printf("\nEnter Available Resources:\n");
    for(i=0;i<m;i++)
        scanf("%d",&av[i]);

    while(c<n) {
        int found=0;

        for(i=0;i<n;i++) {
            if(!f[i]) {

                for(j=0;j<m;j++)
                    if(need[i][j]>av[j]) break;

                if(j==m) {
                    for(k=0;k<m;k++)
                        av[k]+=a[i][k];

                    s[c++]=i;
                    f[i]=1;
                    found=1;
                }
            }
        }
    }
}
```

```

        if(!found) {
            printf("\nSystem is not safe");
            return 0;
        }
    }

    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i=0;i<n;i++) {
        printf("P%d",s[i]);
        if(i<n-1) printf(" -> ");
    }
}

```

OUTPUT

```

Enter number of processes: 3
Enter number of resources: 3

Enter Allocation Matrix:
0
1
0
2
0
0
1
1
1

Enter Maximum Demand Matrix:
1
2
1
2
1
1
1
3
1
1

Enter Available Resources:
1
1
1

System is in a safe state.
Safe sequence is: P0 -> P1 -> P2
Process returned 0 (0x0)   execution time : 28.139 s
Press any key to continue.

```

Code in C Deadlock

```
#include<stdio.h>

int main() {
    int n,m,i,j,k,f[10]={0},a[10][10],r[10][10],av[10],found;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter number of resources: ");
    scanf("%d",&m);

    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&a[i][j]);

    printf("\nEnter Request Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&r[i][j]);

    printf("\nEnter Available Resources:\n");
    for(i=0;i<m;i++)
        scanf("%d",&av[i]);

    do {
        found=0;

        for(i=0;i<n;i++) {
            if(!f[i]) {

                for(j=0;j<m;j++)
                    if(r[i][j]>av[j]) break;

                if(j==m) {
                    for(k=0;k<m;k++)
                        av[k]+=a[i][k];

                    f[i]=1;
                    found=1;
                }
            }
        }
    } while(found);

    int d=0;

    printf("\nProcesses in Deadlock:\n");
```

```

for(i=0;i<n;i++)
    if(!f[i]) {
        printf("P%d ",i);
        d=1;
    }

if(!d)
    printf("No deadlock detected. All processes can finish.");
}

```

OUTPUT

Enter number of processes: 3	Enter number of processes: 3
Enter number of resources: 3	Enter number of resources: 3
Enter Allocation Matrix:	Enter Allocation Matrix:
1	1
0	0
0	0
0	0
1	1
0	0
1	0
1	0
0	1
Enter Request Matrix:	Enter Request Matrix:
0	0
0	1
1	0
1	0
0	0
0	1
0	1
1	0
1	0
Enter Available Resources:	Enter Available Resources:
1	0
1	0
1	0
Processes in Deadlock:	Processes in Deadlock:
No deadlock detected. All processes can finish.	P0 P1 P2
Process returned 0 (0x0) execution time : 25.810 s	Process returned 0 (0x0) execution time : 25.616 s
Press any key to continue.	Press any key to continue.

Lab program 6:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

- a) Worst-fit
- b) Best-fit
- c) First-fit.

Code in C

```
#include <stdio.h>

void allocate(int b[], int m, int p[], int n, int type)
{
    int a[n];

    for (int i = 0; i < n; i++)
        a[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int idx = -1;

        for (int j = 0; j < m; j++)
        {
            if (b[j] >= p[i]) // block can fit process
            {
                if (type == 0) // First Fit
                {
                    idx = j;
                    break;
                }

                if (type == 1) // Best Fit
                {
                    if (idx == -1 || b[j] < b[idx])
                        idx = j;
                }
            }
        }
    }
}
```

```

    }

    if (type == 2) // Worst Fit
    {
        if (idx == -1 || b[j] > b[idx])
            idx = j;
    }
}

if (idx != -1)
{
    a[i] = idx;
    b[idx] = -1; // only ONE process per block
}
}

if (type == 0) printf("\n--- First Fit ---\n");
if (type == 1) printf("\n--- Best Fit ---\n");
if (type == 2) printf("\n--- Worst Fit ---\n");

printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, p[i]);

    if (a[i] != -1)
        printf("%d\n", a[i] + 1);
    else
        printf("Not Allocated\n");
}

```

```

}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int b[m], b1[m], b2[m], b3[m];

    printf("Enter sizes of %d memory blocks:\n", m);

    for (int i = 0; i < m; i++)
    {
        scanf("%d", &b[i]);
        b1[i] = b2[i] = b3[i] = b[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int p[n];

    printf("Enter sizes of %d processes:\n", n);

    for (int i = 0; i < n; i++)
        scanf("%d", &p[i]);

    allocate(b1, m, p, n, 0);
    allocate(b2, m, p, n, 1);
    allocate(b3, m, p, n, 2);

```

```
    return 0;
}
```

OUTPUT

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.      Process Size      Block No.
1                212                2
2                417                5
3                112                3
4                426                Not Allocated

--- Best Fit ---
Process No.      Process Size      Block No.
1                212                4
2                417                2
3                112                3
4                426                5

--- Worst Fit ---
Process No.      Process Size      Block No.
1                212                5
2                417                2
3                112                4
4                426                Not Allocated

Process returned 0 (0x0)   execution time : 31.897 s
Press any key to continue.
|
```

Lab program 7:

Write a C program to simulate page replacement algorithms. (Any one)

- a) FIFO
- b) LRU
- c) Optimal

Code in C

```
#include <stdio.h>

int fifo(int p[], int n, int f) {
    int fr[10], i, j, k = 0, fault = 0, found;

    for(i = 0; i < f; i++) fr[i] = -1;

    for(i = 0; i < n; i++) {
        found = 0;
        for(j = 0; j < f; j++)
            if(fr[j] == p[i]) found = 1;

        if(!found) {
            fr[k] = p[i];
            k = (k + 1) % f;
            fault++;
        }
    }
    return fault;
}

int lru(int p[], int n, int f) {
    int fr[10], time[10], i, j, pos, min, fault = 0;

    for(i = 0; i < f; i++) {
```

```

        fr[i] = -1;
        time[i] = -1;
    }

    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < f; j++) {
            if(fr[j] == p[i]) {
                found = 1;
                time[j] = i;
            }
        }

        if(!found) {
            min = time[0];
            pos = 0;

            for(j = 1; j < f; j++) {
                if(time[j] < min) {
                    min = time[j];
                    pos = j;
                }
            }

            fr[pos] = p[i];
            time[pos] = i;
            fault++;
        }
    }

    return fault;
}

```

```

int optimal(int p[], int n, int f) {

```

```
int fr[10], i, j, k, pos, farthest, fault = 0;
```

```
for(i = 0; i < f; i++)
```

```
    fr[i] = -1;
```

```
for(i = 0; i < n; i++) {
```

```
    int found = 0;
```

```
    for(j = 0; j < f; j++) {
```

```
        if(fr[j] == p[i]) {
```

```
            found = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
if(!found) {
```

```
    pos = -1;
```

```
    farthest = -1;
```

```
for(j = 0; j < f; j++) {
```

```
    int nextUse = -1;
```

```
    for(k = i + 1; k < n; k++) {
```

```
        if(fr[j] == p[k]) {
```

```
            nextUse = k;
```

```
            break;
```

```
        }
```

```
    }
```

```
if(nextUse == -1) {
```

```
    pos = j;
```

```
    break;
```

```

        }

        if(nextUse > farthest) {
            farthest = nextUse;
            pos = j;
        }
    }

    fr[pos] = p[i];
    fault++;
}
}

return fault;
}

int main() {
    int p[20], n, f, i;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter pages: ");
    for(i = 0; i < n; i++)
        scanf("%d", &p[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    printf("\nFIFO Faults    = %d", fifo(p, n, f));
    printf("\nLRU Faults    = %d", lru(p, n, f));
    printf("\nOptimal Faults = %d", optimal(p, n, f));
}

```

OUTPUT

```
Enter number of pages: 10
Enter pages: 1 2 1 3 1 2 4 5 2 1
Enter number of frames: 3

FIFO Faults    = 7
LRU Faults     = 6
Optimal Faults = 5
Process returned 0 (0x0)   execution time : 50.125 s
Press any key to continue.
|
```